EcoStream Production Infrastructure Runbook

This runbook describes the operational procedures, deployment workflow, disaster recovery plan, and scaling strategy for the EcoStream production infrastructure. The platform is designed using AWS best practices with environment isolation, containerized deployments, and automated CI/CD pipelines.

1. Architecture Overview

- Three isolated environments: Development, Test, and Production.
- Each environment runs inside its own VPC within the same AWS account.
- Application containers run on Amazon ECS (Fargate).
- Container images are stored in Amazon ECR.
- Traffic is routed through an Application Load Balancer.
- The application connects to a managed PostgreSQL database on Amazon RDS.
- Monitoring and alerting are handled using Amazon CloudWatch.

2. Deployment Flow – How we deploy on a Friday without fear

1. Developers push application code to the GitHub repository.
2. GitHub Actions triggers the CI/CD pipeline automatically.
3. The pipeline runs basic checks and builds a Docker image for the application.
4. The Docker image is pushed to Amazon Elastic Container Registry (ECR).
5. Amazon ECS pulls the new image from ECR.
6. ECS performs a rolling deployment to replace old containers with new ones.
7. Application Load Balancer health checks ensure only healthy containers receive traffic.
8. If deployment fails health checks, ECS automatically rolls back.

3. Disaster Recovery – What happens if the database vanishes?

- The database runs on Amazon RDS with Multi-AZ deployment.
- Automated backups are enabled with daily snapshots.
- Transaction logs allow point-in-time recovery.
- If the primary database instance fails, RDS automatically fails over to the standby instance.

Recovery Procedure:

1. Identify the latest healthy backup or snapshot.
2. Restore a new RDS instance from the snapshot.
3. Update the ECS application configuration if needed.

4. Validate application connectivity and restore traffic.

4. Scaling Strategy – Handling 10x traffic growth

- ECS Service Auto Scaling increases the number of running containers based on CPU or memory usage.
- Application Load Balancer distributes traffic across containers.
- RDS supports storage auto scaling and can be extended with read replicas for read-heavy workloads.
- CloudFront CDN can be introduced to cache static content and reduce backend load.

5. Observability and Monitoring

- Amazon CloudWatch collects infrastructure and application metrics.
- CloudWatch Logs aggregates application logs from ECS containers.
- Alarms notify the operations team when thresholds are exceeded.
- Metrics monitored include CPU utilization, memory usage, error rate, and response latency.

6. Security and Secrets Management

- Sensitive credentials are stored in AWS Secrets Manager.
- ECS tasks access secrets using IAM roles instead of hardcoded credentials.
- Security groups restrict access between load balancer, application containers, and database.
- Database instances are deployed in private subnets and are not accessible from the public internet.

7. Environment Isolation Strategy

- Development, Test, and Production environments run in separate VPCs.
- This prevents accidental interference between environments.
- Network segmentation ensures production databases and services remain isolated.
- Separate IAM policies restrict access to production resources.